

Introduction to Operating Systems

What is an Operating System?
Functions of Operating System
Layers of Operating System
Types of Operating System

Dr. Manmath Narayan Sahoo
Department of Computer Science & Engineering
National Institute of Technology Rourkela
Odisha – 769008

Disclaimer – This slide is prepared taking content from various books and online resources. The presenter doesn't own the content. The inherited content is used for educational purpose only.

Recommended Books

- **Operating System Concepts** by Abraham Silberschatz, Peter B. Galvin and Greg Gagne
- **Operating Systems: Internals and Design Principles** by William Stallings

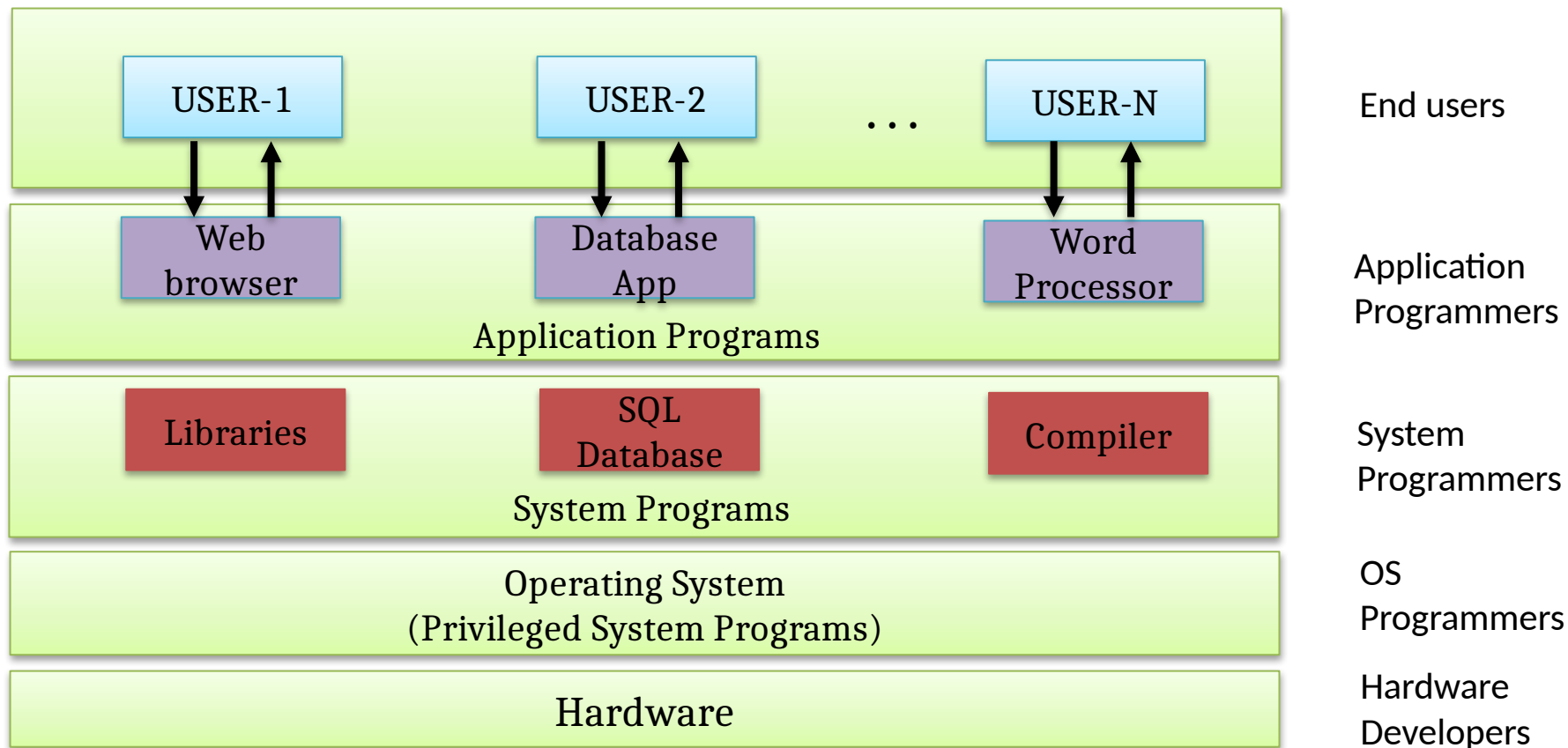
Evaluation

- Mid Term: 30%
- End Term: 50%
- TA through Assignments and Quiz Tests: 20%

Introduction

- Consider a bare machine with hardware only.
 - Can we write a simple program and execute it on h/w directly; without any s/w?
 - **YES**
 - Why do we need software then?
 - **Convenience**
 - **It abstracts the underlying implementation details**
- OS is an interface between the user and the hardware.
- Operating System (OS) is a collection of software that manages hardware.

Abstract view of a Computer System



Abstract view of a Computer System

- 2 view points of OS
 - USER View (Convenience)
 - SYSTEM View (many complicated tasks that are abstracted from users)

Functions of OS

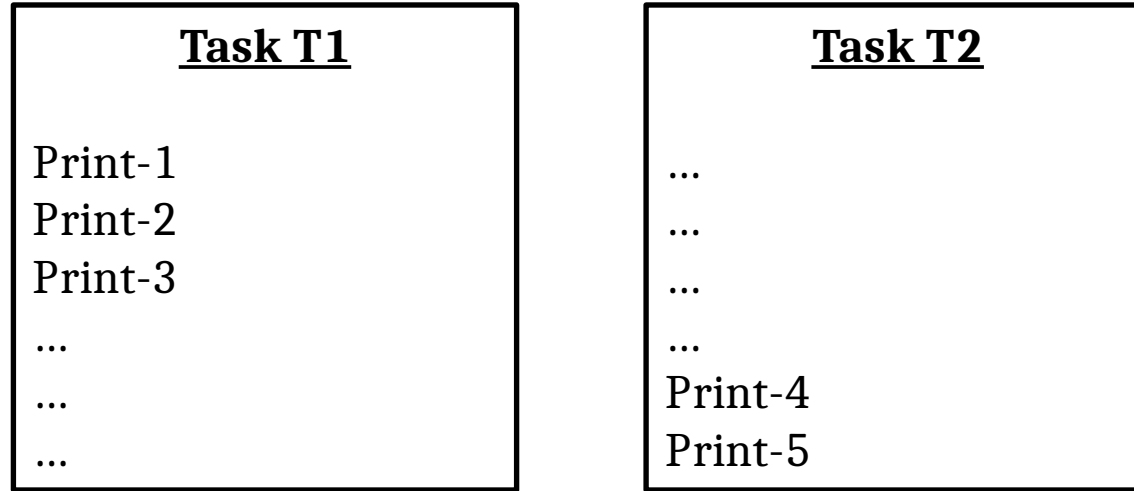
- Interface: for easy access to hardware
- Server: Provides services
- Resource Manager:
 - CPU: OS assigns CPU to different tasks being executed.
 - Main memory: Processes are to be stored.
 - Secondary Memory: Programs/Files are to be stored.
 - I/O devices: Co-ordination and assignment of different I/O devices among the executing processes.

Functions of OS

- Accountant: Keeps track of
 - How much memory is used; and how much is free.
 - How much time a user has to use a printer or CPU, etc.
- Guardian to the System: Protects all resources. E.g., A printer may not be used by all users.
- Coordinator: If T_i need the input from T_j then T_i can't start its execution before T_j .

Functions of OS

- Resource Utilization Maximizer:
 - Printer is to be allocated to T1 first.
 - By the time T2 demands for printer, T1 has already used it



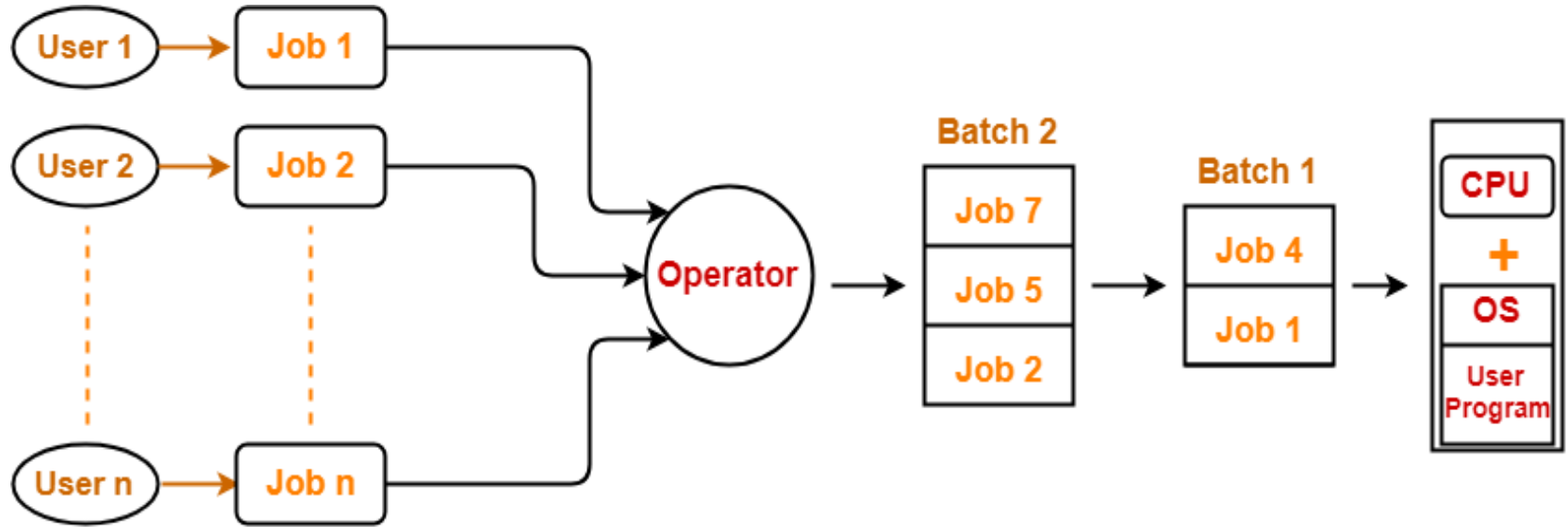
OS: Layered Structure

Shell
Long Term Scheduler (LTS)
Short Term Scheduler (STS)
Resource Manager
File Manager
Memory Manager
I/O Control System (IOCS)
KERNEL
Hardware

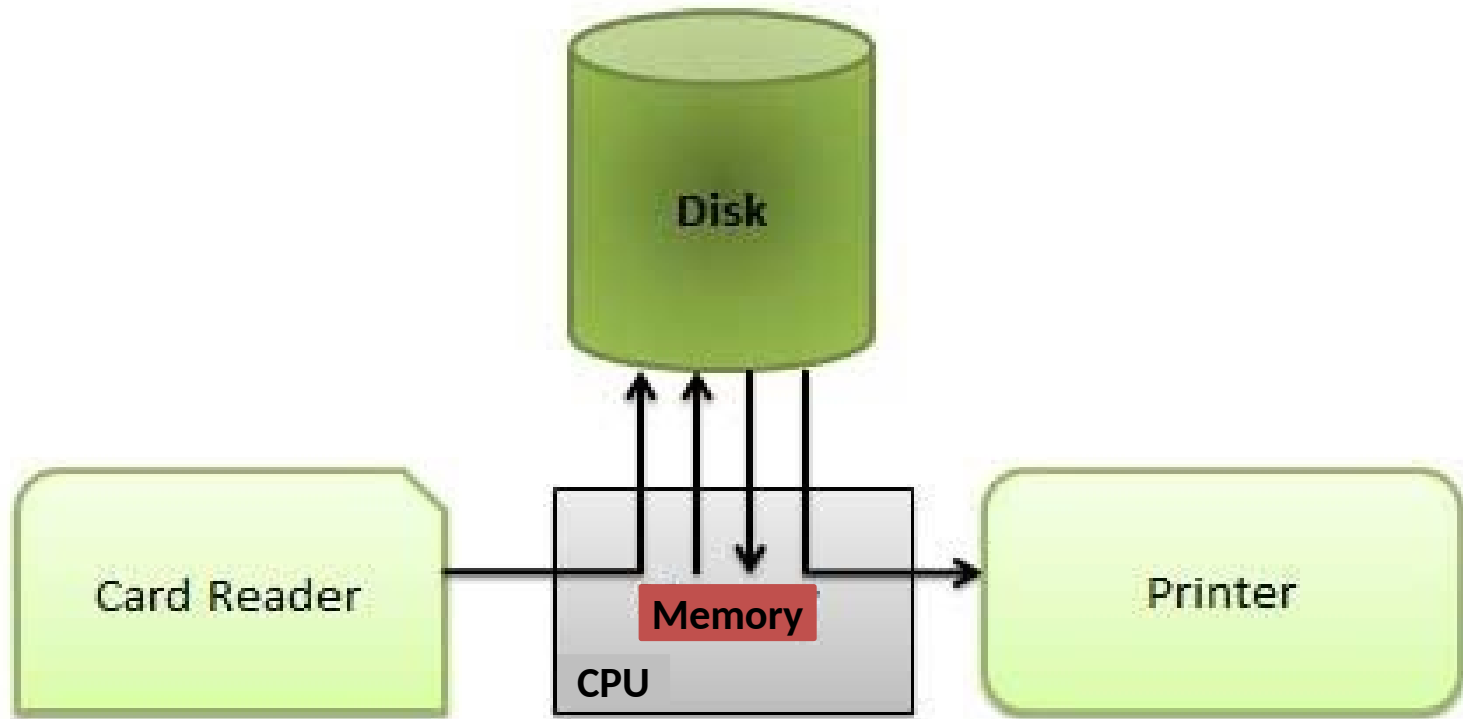
Types of OS

- Batch OS
- Multi Programming OS
- Multi Processing OS
- Time Sharing OS
- Multi Threading OS
- Real Time OS

Batch OS



Batch OS: **SPOOL**ing

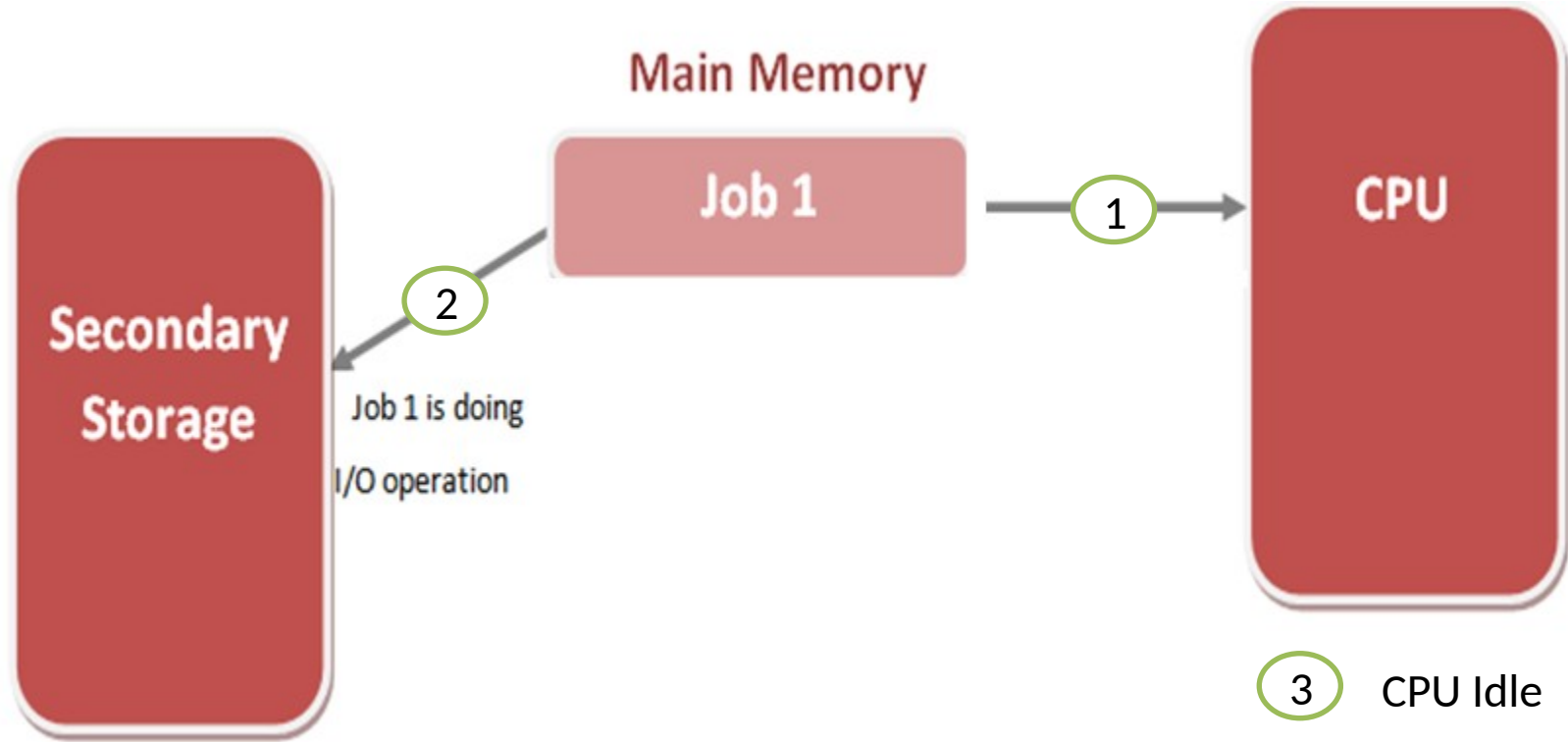


(**S**imultaneous **P**eripheral **O**peration **O**n**L**ine)

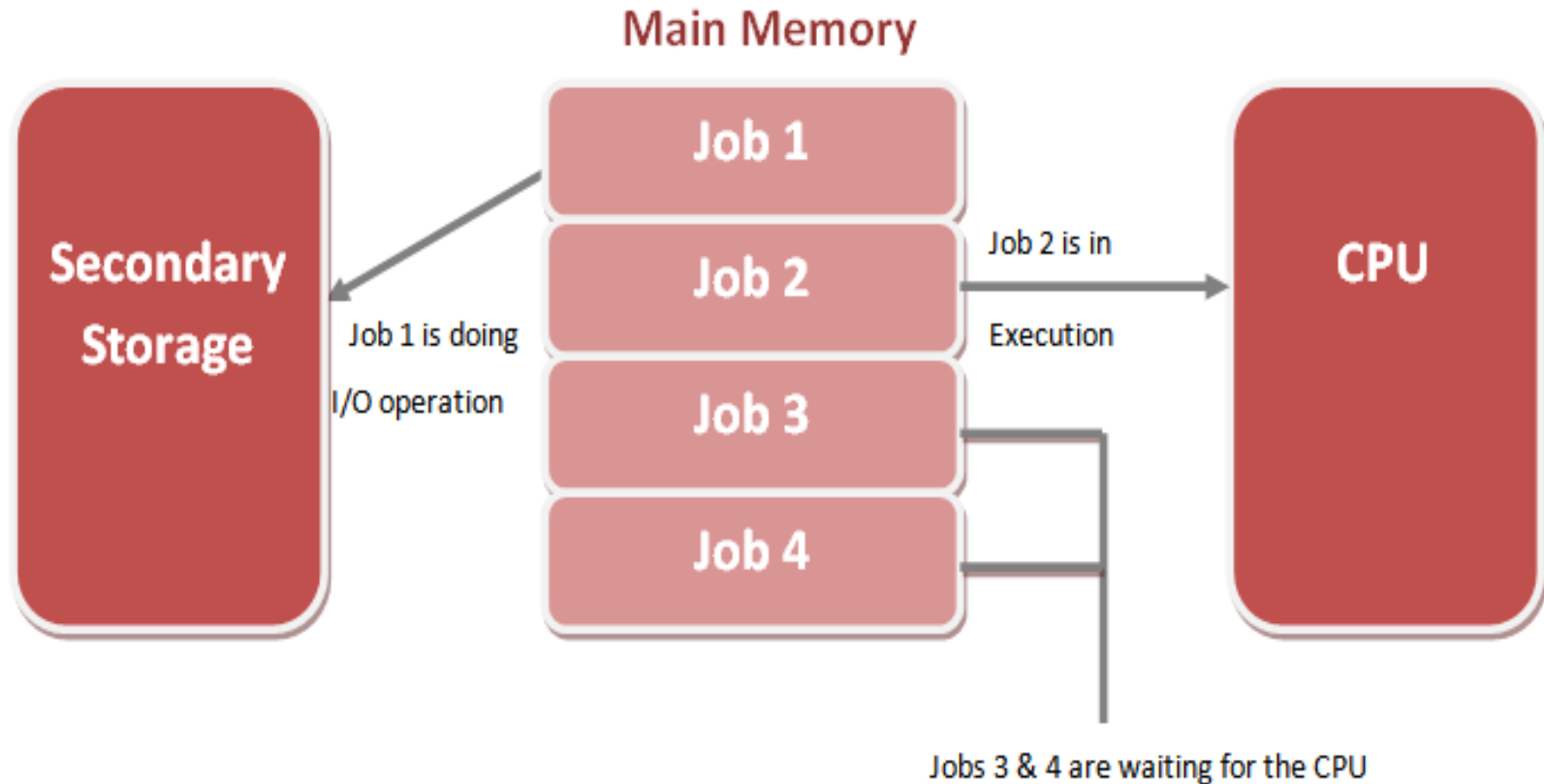
Batch OS: SPOOLing

- No interaction of I/O devices with CPU. CPU directly interacts with Disk which is faster.
- Input of one job, output of a second job and processing of a third job can be overlapped.
- Disadvantage:
 - Suitable for non-interactive jobs
 - Execution was based on uniprogramming model

UniProgramming



Multiprogramming OS



Multiprogramming OS

- jobA = 200ms execution + 30ms i/o at 100th ms
- jobB = 100ms execution + NO i/o



- CPU idle time = 30ms

Job	Turn-around time	Response time
A	230	0
B	330	230

Multiprogramming OS

- jobA = 200ms execution + 30ms i/o at 100th ms
- jobB = 100ms execution + NO i/o



- CPU idle time = 0

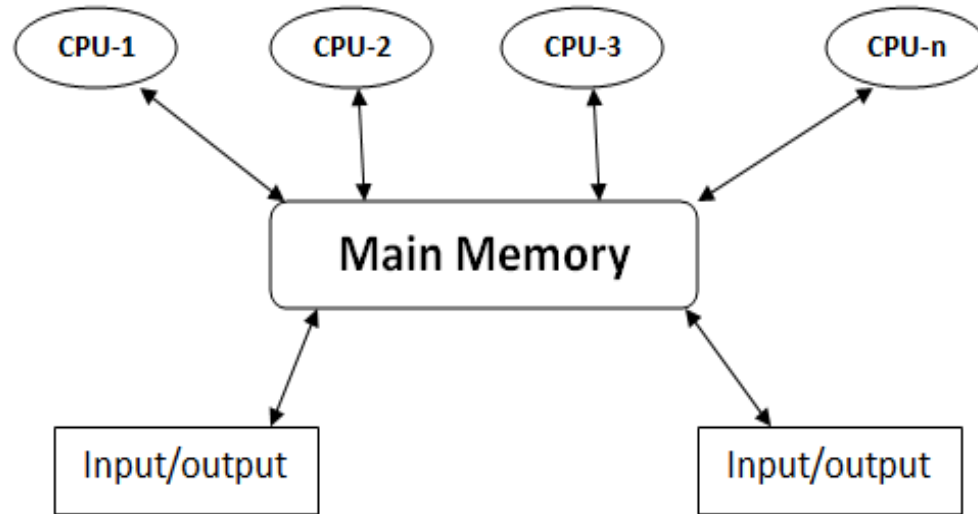
Job	Turn-around time	Response time
A	230	0
B	300	100

Multiprogramming OS

- Advantages
 - Efficient CPU utilization.
 - User feels that many programs are allotted CPU almost simultaneously.
 - Less response time.
 - Useful when load is more.
- Difficulties
 - Complex CPU scheduling.
 - Memory management is required to accommodate many jobs

Multiprocessing OS

- Multiple **processors** can execute different processes OR multiple independent instructions of same process

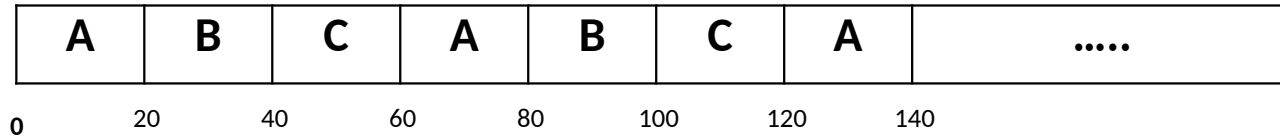


Multiprocessing OS

- Advantages
 - Increased Throughput
 - Increased Reliability.
 - True Parallel Processing.
- Difficulties
 - More complex – Scheduling, synchronization.

Time Sharing OS

- Predefined time quantum **q=20ms** (say)
- Processes are executed in FCFS order, none exceeding the quantum in a turn



Time Sharing OS

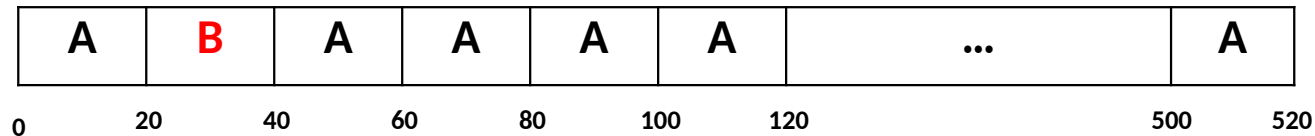
job-A: 500ms + NO i/o

job-B: 20ms + NO i/o

- Without time sharing



- With time sharing

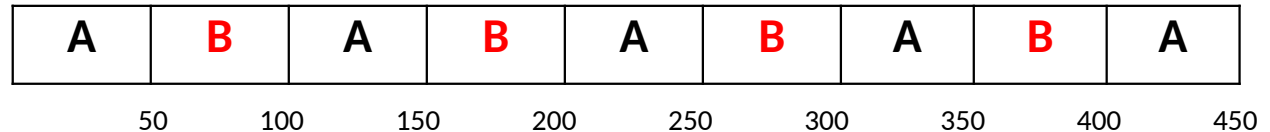


	Without TS	With TS
Response time	A=0, B=500	A=0, B=20
Turn-around time	A=500, B=520	A=520, B=40

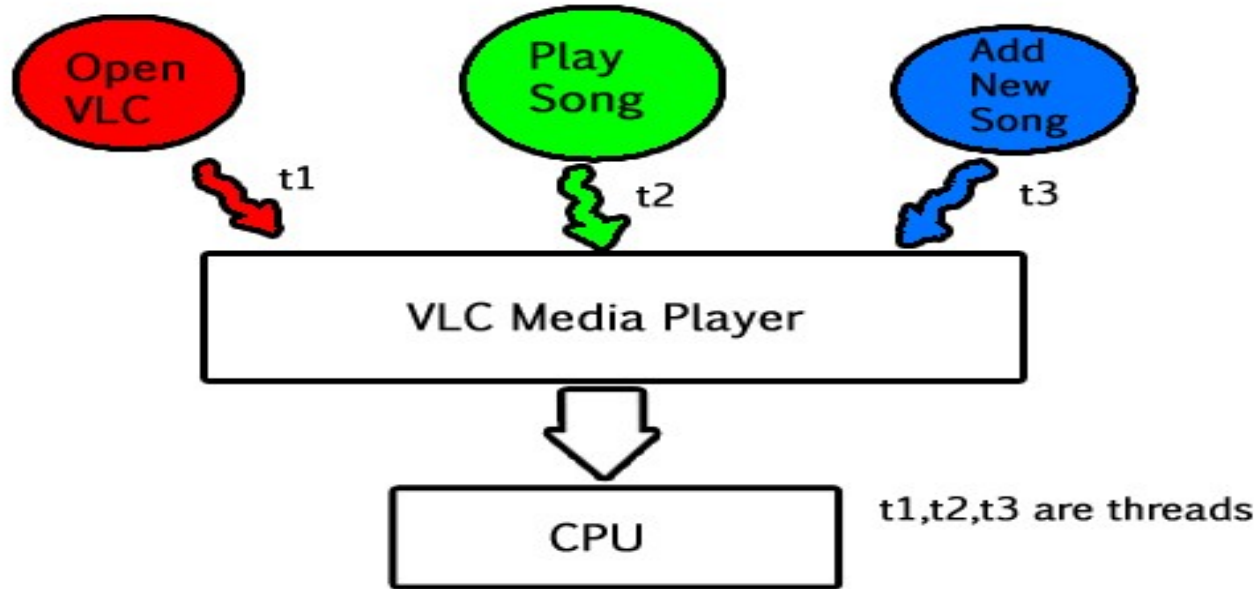
Time Sharing OS

- Advantages
 - quick response.
 - reduces CPU idle time.
 - Disadvantages
 - Switching overhead
 - More turn-around time
- **A**: 250ms; **B**: 200ms, $q=50$ ms

	Resp. time	TA time
A	0	450
B	50	400



Multi-Threading OS



Multi-Threading OS

- Advantages:
 - Max utilisation of CPU is possible.
 - Time Saving.
 - Improves the performances of complex applications.
 - Parallelizes tasks.
- Difficulties:
 - Complex debugging and testing process.
 - Increased complexity level in writing a program.

Real Time OS

- Real time OSs are specifically designed to execute *real time tasks*.
- Real time tasks are **deadline** constrained
- 3 types depending on the consequences of a task missing deadline
 - Hard real time task
 - Firm real time task
 - Soft real time task

Real Time OS

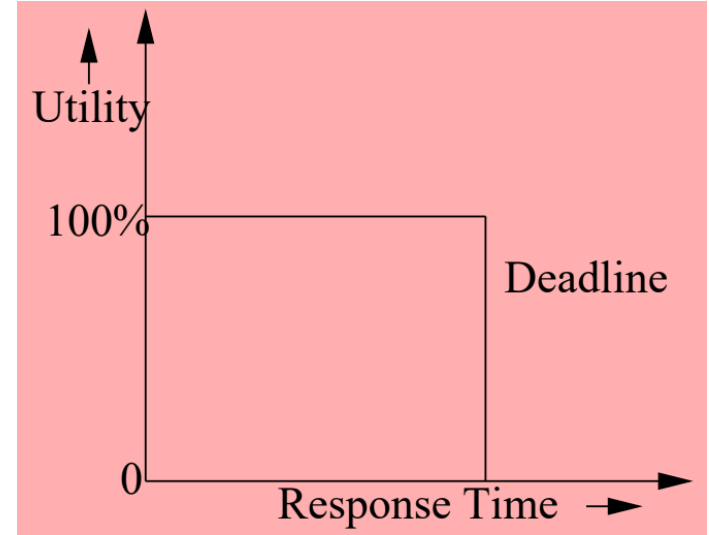
Hard real time tasks

- The task and the system is considered to be failed if deadline is not met.
- Example
 - obstacle detection by a robot
 - anti-missile system

Real Time OS

Firm real time tasks

- If a firm RT task fail to meet deadline then the system does not fail.
- Late results are simply discarded
- Example
 - Video Conferencing
 - Satellite based enemy territory tracking



Real Time OS

Soft real time tasks

- Deadlines are not absolute, in contrast to Hard and Firm RT tasks
- Time constraints are expressed in terms of average response time
- Missed deadlines of soft RT tasks does not imply system failure
- After the deadline, the utility of the results falls continuously
- Example: Web browsing, Railway ticket reservation

